---

title: ”VitalChronicle User Manual” author: ”Sebastiano Romi” date: ”Updated for VitalChronicle 1.1.8” lang: en-US geometry: margin=2.2cm colorlinks: true linkcolor: blue urlcolor: blue toc: true toc-depth: 3

About this manual

This is the authoritative user manual for **VitalChronicle 1.1.8**. VitalChronicle is a local-first desktop dashboard for personal Google Health data and optional local AI analysis through Ollama.

VitalChronicle is independent from Google and is not a medical device. Charts, statistical bands, anomaly indicators, correlations, and language-model output are exploratory. They do not diagnose disease, establish causality, or replace validated measurements and professional care.

VitalChronicle is and will remain free and open source. Voluntary support through Buy Me a Coffee helps maintain the project.

Interface language

VitalChronicle reads the desktop and process locale at startup, including the Linux `LANGUAGE`, `LC_ALL`, `LC_MESSAGES`, and `LANG` settings. English and Italian are included; when no supported language is detected, the interface and local-AI answers use English.

Use **Settings → Language** to select **System default** or a supported language manually. The choice is stored for future launches; restart VitalChronicle once after changing it. Advanced users and screenshot automation can temporarily override both automatic detection and the saved choice:

```
VITALCHRONICLE_LANGUAGE=it vitalchronicle
```

VitalChronicle checks for a new public release shortly after startup without blocking the interface and repeats the check once every hour while the application remains open. Health data are never included in the request. To check immediately, choose **Help → Check for updates**. When a newer release is available, the dialog distinguishes maintenance, feature, and major updates and can open the official GitHub release page in the system browser. Portable AppImage and Windows builds also offer **Update now** when the release contains the exact matching package. VitalChronicle downloads that asset beside the running file, verifies its SHA-256 checksum, replaces the file at the same path, and keeps the previous build with a `.previous` suffix. If postponed, the same release is not shown again for ten days.

The badge next to the VitalChronicle title always shows the installed version. It turns amber and adds an upward arrow plus the latest version when an update is available. Selecting the normal badge starts an immediate check; selecting an update badge opens the official release. More precisely, the update badge refreshes the release metadata and opens the available update options. Python/virtual-environment and macOS installations use the release page because replacing those layouts automatically would not be reliably safe.

Additional languages are maintained by the community on Weblate and are included in releases after automatic catalogue validation.

1. What VitalChronicle does

VitalChronicle can:

- guide the user through personal Google Cloud and OAuth configuration;
- download the categories exposed by the Google Health API;
- retain a private, incremental SQLite archive on the computer;
- update missing intervals instead of downloading the complete history repeatedly;
- refresh automatically at startup and every ten minutes while running;
- visualise each metric with an appropriate chart;
- compare current values with a seven-day personal baseline;
- export individual categories as CSV or the complete archive as ZIP/JSON;
- deterministically prepare personal baselines, trends, anomalies, data-quality limits, structured sleep/workout comparisons, and cautious cross-metric associations;
- run a two-pass complete-history analysis or answer questions about a selected period;
- maintain private, persistent AI conversations with follow-up questions and pinned data;
- show the model's thinking while it runs and replace it with the final answer in the same assistant area.

VitalChronicle does not upload health records to the developer or to a hosted AI provider. Google receives authenticated API requests, and local AI requests are sent to the Ollama service on `127.0.0.1`.

2. System requirements

2.1 Packaged applications

Platform	Requirement
Linux	x86-64 distribution with FUSE support for AppImage, or AppImage extract-and-run
Windows	64-bit Windows 10 or 11

Platform	Requirement
macOS Apple Silicon	Recent arm64 macOS release
macOS Intel	Recent x86-64 macOS release

Internet access is required for Google authentication and synchronisation. AI analysis does not require Internet after the Ollama model has been downloaded.

2.2 Running from source

- Python 3.10 or newer;
- PySide6 and the dependencies declared in `pyproject.toml`;
- a desktop session able to run Qt applications;
- optional Ollama for local AI.

2.3 Local AI hardware profiles

VitalChronicle provides two initial profiles:

- **NVIDIA GPU · 16 GB RAM** — `qwen3.5:9b` is the balanced recommendation;
- **CPU only · 32 GB RAM** — `qwen3:30b-a3b` is the balanced recommendation.

Other installed Ollama model names can be entered manually. Larger models can be very slow or may require more memory than the computer can provide.

3. Installation

3.1 Linux AppImage

1. Download the Linux AppImage from the latest release.
2. Make it executable:

   ```
   chmod +x VitalChronicle-1.1.8-linux-x86_64.AppImage
   ```

3. Start it:

   ```
   ./VitalChronicle-1.1.8-linux-x86_64.AppImage
   ```

If FUSE is unavailable, run it with:

```
APPIMAGE_EXTRACT_AND_RUN=1 ./VitalChronicle-1.1.8-linux-x86_64.AppImage
```

The AppImage is accompanied by a Sigstore bundle and release checksums.

3.2 Windows

1. Download `VitalChronicle-1.1.8-windows-x86_64.exe`.
2. Verify its SHA-256 checksum against `SHA256SUMS.txt`.
3. Start the executable.

Windows SmartScreen may warn because the file is not signed with a paid Microsoft-trusted certificate. Downloading from the official repository and matching the checksum confirms that the file is the one produced by the public workflow.

3.3 macOS

Choose the correct DMG:

- `arm64` for Apple Silicon;
- `x86_64` for Intel Macs.

Open the DMG and launch VitalChronicle. The application is ad-hoc signed but not notarised through the paid Apple Developer programme, so Gatekeeper may require explicit approval from **System Settings → Privacy & Security**.

3.4 Fedora source installation

```
sudo dnf install python3 python3-pip
git clone https://github.com/SebRoLENS/google-health-dashboard-ai.git
cd google-health-dashboard-ai
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
vitalchronicle
```

3.5 Windows source installation

```
py -3 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e .
vitalchronicle
```

3.6 macOS source installation

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
vitalchronicle
```

4. Google Cloud and OAuth setup

Each user creates a personal OAuth client. The repository deliberately does not contain a shared client secret. This keeps control of the credentials with the person whose health data are being accessed. No programming knowledge is required: choose **Google setup** in VitalChronicle and keep the seven-step wizard open while following the browser instructions.

Before starting, prepare:

- the Google account containing the health data;
- an internet connection and a normal system browser;
- about ten minutes;
- a secure place for the downloaded OAuth JSON file.

Never publish that JSON, attach it to an issue, or send it to the developer. It contains a client secret for the personal Google Cloud project.

4.1 Create a project

1. In wizard step 1, click **Open project selector**.
2. Sign in with the Google account containing the health data.
3. In Google Cloud, click **New project**.
4. Enter `VitalChronicle Personal` as the project name.
5. If a **Location** field appears, choose **No organisation** for a normal personal account.
6. Click **Create** and wait for the project to be prepared.
7. Select the new project and confirm that its name appears in the Google Cloud top bar.

The project is only a container for API and OAuth configuration. Creating it does not upload the local VitalChronicle archive and normally does not require enabling billing.

4.2 Enable Google Health API

1. In wizard step 2, click **Open Google Health API**.
2. Check the project selector in the Google Cloud top bar. It must show the project created above.
3. Click **Enable**.
4. Wait until the page shows **Manage** or otherwise confirms that the API is enabled.

If the button already says **Manage**, the API is already enabled and no further action is needed on that page.

4.3 Configure Branding and Audience

In wizard step 3, first click **Open Branding**. If the Google Auth Platform has not yet been configured, click **Get started** and enter:

1. **App name:** `VitalChronicle Personal`.
2. **User support email:** the user's own Google address.
3. **Audience: External**.
4. **Contact email:** the user's own address.
5. Accept the policy acknowledgement if it is displayed, then click **Create**.

Next, click **Open Audience / Test users**:

1. Confirm **Publishing status: Testing** and **User type: External**.
2. Find **Test users** and click **Add users**.
3. Enter the exact Google address containing the health data.
4. Click **Save**.
5. Confirm that the address now appears in the Test users list before continuing.

Testing is not mandatory. It is normally the simplest personal-use configuration and does not require public verification. Testing refresh tokens normally expire after seven days; VitalChronicle will then ask the user to authenticate again. Production can require Google verification and additional compliance work for sensitive or restricted scopes.

4.4 Add Data Access scopes

In wizard step 4:

1. Click **Open Data Access**.
2. Click **Add or remove scopes**.
3. In the API filter, search for `Google Health API`.
4. Select the required read-only scopes. For the complete dashboard, select all scopes listed below; the wizard's **Copy all read-only scopes** button copies them exactly.
5. Click **Update** at the bottom of the scope panel.
6. Back on the Data Access page, click **Save**.

The current read-only groups are:

```
https://www.googleapis.com/auth/googlehealth.activity_and_fitness.readonly
https://www.googleapis.com/auth/googlehealth.health_metrics_and_measurements.readonly
https://www.googleapis.com/auth/googlehealth.sleep.readonly
https://www.googleapis.com/auth/googlehealth.nutrition.readonly
https://www.googleapis.com/auth/googlehealth.ecg.readonly
https://www.googleapis.com/auth/googlehealth.irn.readonly
https://www.googleapis.com/auth/googlehealth.profile.readonly
https://www.googleapis.com/auth/googlehealth.settings.readonly
https://www.googleapis.com/auth/googlehealth.location.readonly
```

You can authorise fewer groups if you do not want the application to request every category. A denied scope simply makes the corresponding categories unavailable.

4.5 Create and import the OAuth client

In wizard step 5:

1. Click **Open OAuth clients**.
2. Click **Create client**.
3. For **Application type**, select **Web application**. Do not select Desktop application.
4. Enter `VitalChronicle personal desktop` as the client name.
5. Leave **Authorised JavaScript origins** empty.
6. Under **Authorised redirect URIs**, click **Add URI**.
7. Use the wizard's **Copy local redirect URI** button and paste exactly:

   `http://localhost:8765/`
8. Check the spelling, `http`, port `8765`, and final slash.
9. Click **Create**.
10. Immediately download the JSON credentials file.
11. Return to VitalChronicle and click **Select downloaded JSON**.
12. Continue only after the wizard displays **Valid OAuth client. The redirect URI is correct.**

Google may only show the complete client secret at creation time. Keep the JSON private. Never commit it to Git, attach it to an issue, or send it to the developer.

If the wizard rejects the file, read the specific message. The most common cause is a missing final slash or adding the URI under JavaScript origins instead of Authorised redirect URIs.

4.6 Choose the data groups

Wizard step 6 lists the data groups that VitalChronicle can request. Leave all categories selected for the complete dashboard and full-history AI analysis. Deselecting a category is safe, but its charts and AI context will be unavailable. No write permission is requested.

4.7 Authenticate in the browser

1. In wizard step 7, confirm that the Google test-user account is ready.
2. Click **Finish and sign in with Google**.
3. The system browser should open automatically. VitalChronicle also displays a small window containing **Open browser** and **Copy sign-in link** as fallbacks.
4. Select the same Google account added under **Test users**.
5. Google may display **This app has not been verified** for a private Testing project. Verify that the project and app name are the ones just created, then use **Advanced → Go to VitalChronicle Personal** to continue.
6. Review the read-only permissions and click **Continue** or **Allow**.
7. Wait until the browser says **Authentication completed**.
8. Close the browser tab and return to VitalChronicle. The first update starts automatically.

The browser redirects only to the loopback server on the same computer. Port `8765` must be free during sign-in. The wizard checks the port before it finishes; the callback is local and is not exposed to the internet. Do not close VitalChronicle during authentication.

5. Synchronisation and local storage

5.1 First download

Choose a period in the top bar and select **Download / update**. The first download can take time because many data categories are paginated independently.

One failed or unavailable category does not stop the others. VitalChronicle completes the remaining downloads and reports warnings at the end.

5.2 Incremental updates

The database stores completed date ranges for each category. Later synchronisations ask Google only for uncovered intervals. The current day remains refreshable so evolving totals can be replaced without duplicating daily roll-ups.

When credentials are available, VitalChronicle attempts a silent incremental refresh:

- shortly after startup;
- every ten minutes while the application remains open.

5.3 Food catalogues

Public food and measurement-unit catalogues are not part of automatic synchronisation. The personal nutrition log remains supported. This prevents large catalogue downloads from blocking health-data updates.

5.4 Storage locations

VitalChronicle keeps the historical internal `GoogleHealthViewer` directory identifier so existing 0.2.12 archives survive the 1.1.8 upgrade. Typical locations are:

- Linux data: `~/.local/share/GoogleHealthViewer/`;
- Linux configuration: `~/.config/GoogleHealthViewer/`;
- macOS: under `~/Library/Application Support/GoogleHealthViewer/`;
- Windows: under the current user's platform application-data directory.

The database file is named `health_data.sqlite3`. Credentials use the system keyring when possible; a permission-restricted local fallback is used when no supported keyring exists.

6. Overview

The **Panoramica** page compares the current reference day with up to seven preceding days. Missing days are not silently converted to zero; the card reports the number of days that actually contributed to a baseline.

6.1 Completion metrics

Steps, sleep duration, and active-zone minutes use progress bars because a cumulative completion metaphor is meaningful for these quantities. A card can exceed 100% when the current value surpasses the personal average.

6.2 Physiological measurements

Heart rate, HRV, oxygen saturation, respiratory rate, sleep-temperature variation, and weight are not goals to "complete". Their cards show neutral above/below-baseline differences.

The heart-rate mini-chart shows only measurements from the current calendar day. A robust curve uses five-minute medians followed by approximately fifteen-minute smoothing. The seven preceding days provide only the mean and one-standard-deviation band; they are not drawn as overlapping daily traces.

HRV, daily oxygen saturation, respiratory rate, and sleep-temperature variation display the most recent seven available daily values together with their mean and standard- deviation band. Weight always displays the most recent measurement, even when it is not from today, and is formatted to one decimal place.

7. Data explorer

Select **Esplora dati** and choose a category from the left tree. Empty categories are hidden by default and can be displayed with the checkbox.

VitalChronicle chooses a visual encoding according to the data:

- daily bars for steps, energy, workouts, and sedentary duration;
- scatter points and a local trend for dense measurements;
- lines for daily physiological summaries;
- stacked bars for sleep stages, activity intensity, and heart-rate-zone data;
- threshold bands for daily heart-rate zones.

The default vertical scale is calculated robustly from the visible time window so a few extreme values do not flatten the rest of the graph. **Tutti i valori Y** includes every visible extreme. Mouse-wheel zoom and panning operate on the time axis.

The table contains up to 5,000 visible rows and the interactive plot up to 20,000 records. Exports are not restricted by those display limits.

8. Exporting data

8.1 Current category

Select a category and choose **Export CSV**. The CSV contains flattened original fields and is suitable for spreadsheets or independent analysis.

8.2 Complete archive

Choose **Export archive** to create a ZIP containing JSON Lines files for downloaded categories plus account/device resources. Treat exported archives as sensitive health records and store them appropriately.

9. Local AI with Ollama

9.1 Installation checks

Install Ollama from its official package, ensure the service is running, and pull a model:

```
ollama --version
curl -sS http://127.0.0.1:11434/api/version
ollama pull qwen3.5:9b
ollama list
ollama run qwen3.5:9b "Reply only with OK"
```

On Fedora, the service can be inspected with:

```
systemctl status ollama --no-pager -l
journalctl -u ollama -n 100 --no-pager
```

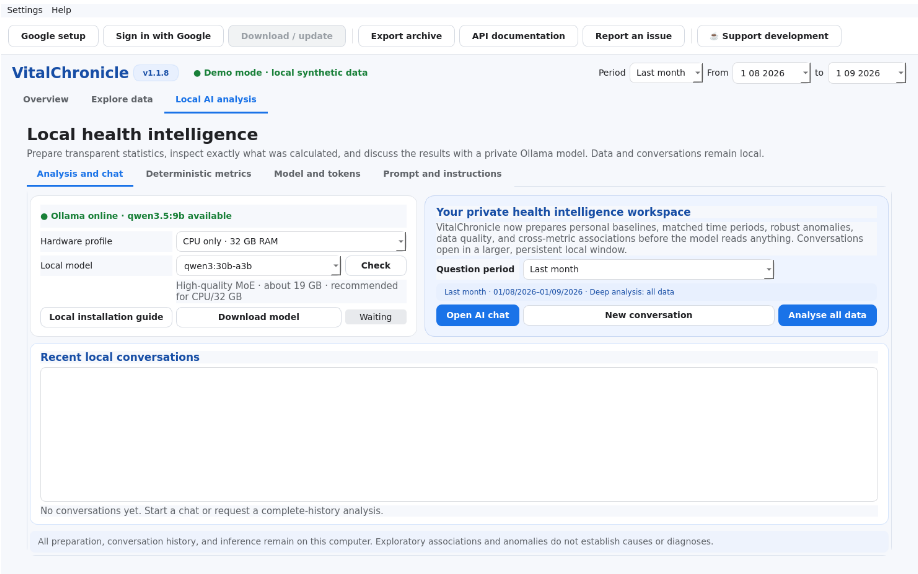
9.2 Hardware profile and model

Choose the closest profile, then select an installed model. VitalChronicle checks the local Ollama endpoint and periodically compares the installed model digest with the registry. Only model metadata are used for that update check; health data are not sent.

9.3 RAM and token recommendation

Open **Local AI analysis → Model and tokens**, enter installed RAM in GB, and request a recommendation. The estimate considers model size, CPU/GPU profile, and the context length reported by Ollama. The token value remains manually editable and is capped only when the model declares a physical context limit. AI settings are intentionally part of the AI workspace rather than a separate application tab.

9.4 AI control centre and conversation window



The **Local AI analysis** tab is one coherent workspace with four internal sections:

Section	Purpose
Analysis and chat	Ollama status, hardware/model choice, question period, recent conversations, a
Deterministic metrics	Requested-versus-observed coverage and every statistic prepared before the mo

Section	Purpose
Model and tokens	RAM-aware output-token recommendation and editable local model settings
Prompt and instructions	The read-only system prompt included with every local-model request

The **Analysis and chat** section provides two main actions:

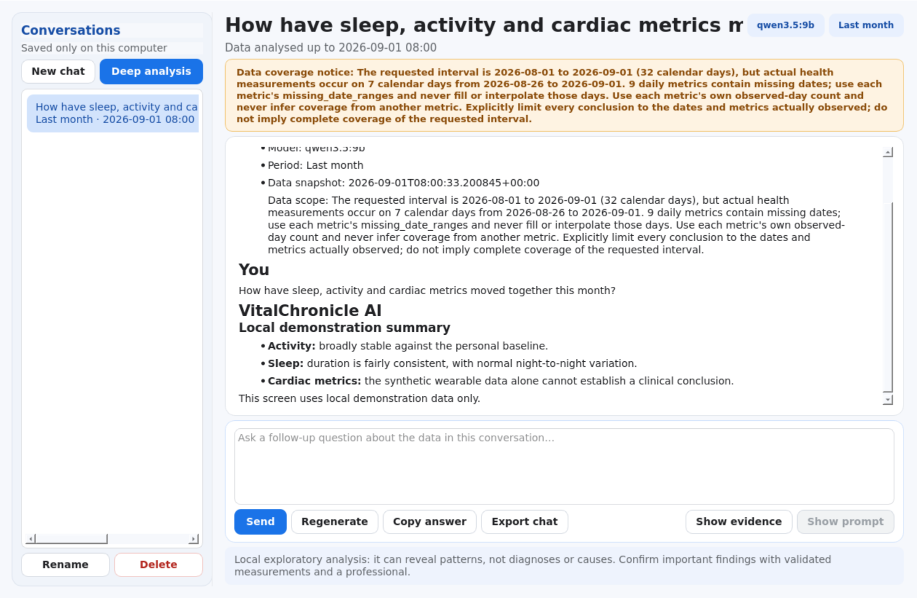
- **Open AI chat** continues the most recent thread or creates one for the selected period;
- **Analyse all data** creates a new thread using every locally available category, regardless of the question-period selector.

For a first analysis:

1. confirm that the status indicator reports Ollama as available;
2. choose the hardware profile and an installed model;
3. open **Model and tokens** if the output-token recommendation needs to be recalculated;
4. choose the period immediately above the question actions when asking a focused question;
5. select **Open AI chat** for that period, or **Analyse all data** to inspect every locally stored category;
6. enter a question and select **Send**;
7. continue with follow-up questions in the same thread so recent dialogue is retained;
8. inspect **Evidence** when an answer needs to be checked against the prepared statistics;
9. select **Show prompt** in chat when the exact messages sent to Ollama need to be audited.

The period selector applies to a new selected-period chat. It does not limit **Analyse all data**. The latter includes secondary numeric fields, sleep stages, workout types, heart-rate zones, body measurements, and any other locally available categories.

9.5 Conversation window, history, and data snapshots



The chat opens in a separate, resizable window so a long answer can be read without compressing the dashboard. Its left sidebar lists locally saved threads, while the main area contains the complete dialogue and one composer for continuing the conversation.

Control	Effect
New chat	Creates an independent selected-period conversation
Deep analysis	Creates a complete-history thread and starts the two-pass analysis
Send	Adds a question or follow-up to the current thread
Stop	Requests cancellation without deleting the existing conversation
Regenerate	Repeats the most recent answer using the same thread and data snapshot
Copy answer	Copies the latest completed answer
Export Markdown	Saves a readable local copy of the dialogue
Evidence	Shows the ranked deterministic evidence available to the model
Show prompt	Reveals the exact latest system/user messages, processed JSON, conversation context
Rename / Delete	Manages only the selected local thread

Each conversation pins a compact processed snapshot, its selected period, model, and data revision. A later Google synchronisation therefore does not silently change the evidence behind an old answer. If the archive becomes newer, the header reports that newer local data exist and offers **Refresh conversation data**. Refreshing replaces the pinned snapshot and records the event in the dialogue.

Recent user and assistant messages are supplied to the model for follow-up continuity. Older turns are compacted into short excerpts so they do not consume the entire model context. Thinking text is transient and is not stored as a second answer.

Above the conversation, a coverage notice distinguishes the requested date interval from the portion actually represented by local records. If the requested month contains only one observed week, the model is required to state that it is analysing that week rather than claiming to analyse a complete month.

The exact-prompt view is read-only and stays on the computer. Because it contains processed health evidence and conversation text, inspect it before copying or sharing it.

Threads and their snapshots are stored locally in `ai_conversations.json`, next to the historical VitalChronicle data archive, with restrictive file permissions where supported. Deleting a thread does not delete health records. The privacy command that removes all local data also clears conversations and credentials.

9.6 Periods and time-of-day handling

New chats and direct questions use the explicit period selector: Today, last seven days, last month, last year, all data, or a custom interval. A complete-history analysis always uses the full local archive.

For totals that accumulate during the day, VitalChronicle compares the partial value with previous days at the same local time when possible. It does not treat an absent current-day value as zero, compare a morning partial total with completed days, or extrapolate that partial total into an entire day. At least three comparable earlier days are required before a same-time difference becomes ranked evidence.

This rule is intended for cumulative metrics such as steps, distance, active minutes, and energy. Physiological measurements such as resting heart rate, HRV, oxygen saturation, and respiratory rate are described as neutral deviations from a personal baseline rather than as goals to complete.

9.7 Deterministic data preparation

Before a health snapshot is sent to local Ollama, Python calculates the evidence that a language model would otherwise have to infer unreliably from raw records:

- personal baselines over 7, 28, and 90 days;
- a matched seven-day period compared with the preceding seven-day period;
- multiweek slope, variability, and day-of-week patterns;
- robust personal anomalies based on the median and median absolute deviation;
- expected daily coverage, missing intervals, and observation recency;

- recent-versus-previous sleep-stage and workout-type summaries;
- same-day and exploratory one-day-lagged associations supported by repeated paired days;
- ranked evidence with explicit confidence and interpretation caveats.

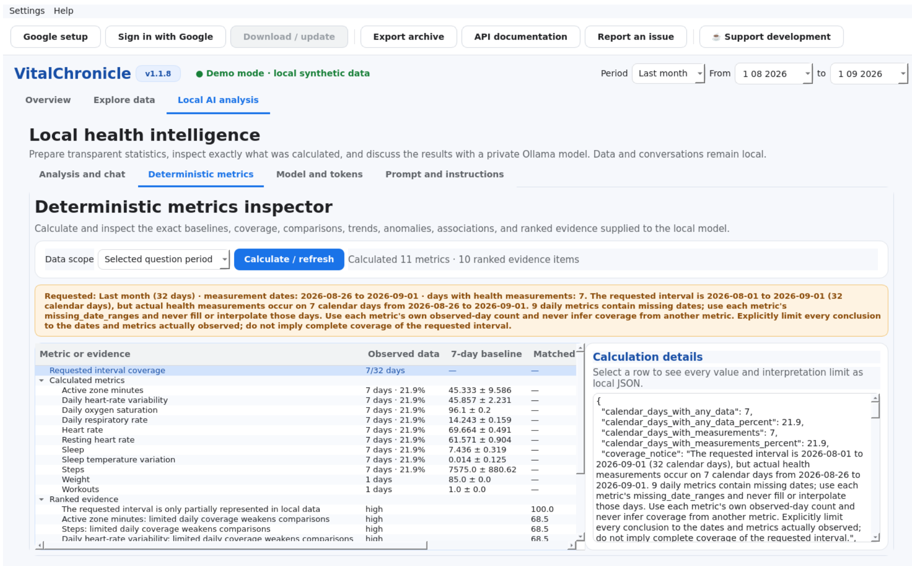
The main evidence families are:

- **Personal baseline — 7, 28, and 90-day windows:** mean, median, standard deviation, minimum, and maximum for the individual user.
- **Matched change — at least three observed days in each adjacent seven-day window:** recent mean versus the immediately preceding equivalent period.
- **Multiweek trend — at least five observed days within 28 days:** direction, weekly slope, and goodness of fit; confidence increases with coverage.
- **Robust anomaly — at least seven observations within 90 days:** median/MAD deviation, which is less sensitive to isolated extremes than mean and standard deviation.
- **Weekly pattern — at least two samples for at least four weekdays:** repeated day-of-week differences rather than one-off daily changes.
- **Coverage and gaps:** expected versus observed daily samples, missing intervals, recency, and the resulting interpretation limits.
- **Same-time comparison — at least three comparable earlier days:** a fair comparison of an incomplete cumulative day at the current local time.
- **Cross-metric association — at least ten paired days:** same-day or exploratory one-day-lagged co-movement, never causality.

Sleep stages are compared over the latest seven days and the preceding seven days. Workout types are summarized over the latest 28 days and the preceding 28 days. Recent heart-rate-zone and active-minute categories are kept as structured totals instead of being flattened into ambiguous prose.

This preparation is descriptive, not diagnostic. A higher or lower value is not labelled as better or worse, associations never imply causality, and missing measurements are never converted to zero.

9.8 Deterministic metrics inspector and interval completeness



Open **Local AI analysis → Deterministic metrics** to calculate and inspect the same snapshot supplied to the local model. Choose either the currently selected question period or the complete local history, then select **Calculate / refresh**. The upper notice reports:

- the requested start, end, and number of calendar days;
- the first and last dates that actually contain a health measurement;
- the number and percentage of requested days represented by health measurements;
- late starts, early endings, gaps, and per-metric expected-versus-observed coverage.

The tree then exposes 7-day baselines, comparable recent/previous changes, trend direction, robust anomaly scores, cross-metric associations, and ranked evidence. Select any row to see its complete JSON, including sample counts and interpretation caveats. A warning about a partly observed interval is itself ranked evidence and is mandatory context for the answer.

This distinction is metric-specific: an irregular body-weight record is not expected every day, while daily totals such as steps can reveal missing calendar coverage. An interval is never considered complete merely because its requested dates span a month or year.

For daily metrics, `missing_date_ranges` records every isolated or consecutive gap in a compact form. Each range includes its first and last date, length, and whether it is leading, internal, or trailing. Up to 64 ranges are included directly;

longer histories report that the range list was truncated. Even minor gaps are kept as mandatory context: 29 observed days out of 32 is not complete, and a second metric with 32 days cannot hide the first metric's three missing dates. The AI is explicitly forbidden to fill or interpolate them.

Google's dated personal heart-rate-zone limits are classified as `reference_configuration`. They describe the thresholds used to interpret other data; they are not physiological measurements. VitalChronicle therefore retains their latest values for context but excludes their dates from measured-day counts and excludes the thresholds from baselines, trends, anomaly detection, and cross-metric associations.

9.9 Ranked evidence and deep synthesis

Potential observations are scored by magnitude, repeated support, coverage, and statistical reliability. At most 20 candidate insights are included, with the strongest evidence IDs prioritized for synthesis. Low-coverage warnings can outrank an apparent trend because a missing-data limitation may be more important than the numerical change itself.

Complete-history analysis uses two model passes: an evidence-selection pass rejects weak or redundant claims, then a synthesis pass writes the user-facing answer. Direct questions use the same prepared evidence with conversational history and a faster single synthesis.

The deterministic JSON is attached to the latest user message in each model call. This is important because an overfull chat context is normally shortened from its older content; placing evidence immediately beside the current request prevents the question from becoming separated from its data. Repeated copies of candidate evidence are removed, but every calculated metric, structured detail, period comparison, association, and coverage limit is retained.

VitalChronicle reads the physical context length from Ollama and remembers it for the selected model. It estimates the input size before inference and, if input plus the requested response cannot fit together, preserves the evidence while reducing the maximum response tokens. The exact estimated input, context size, and response allowance appear under **Show prompt**.

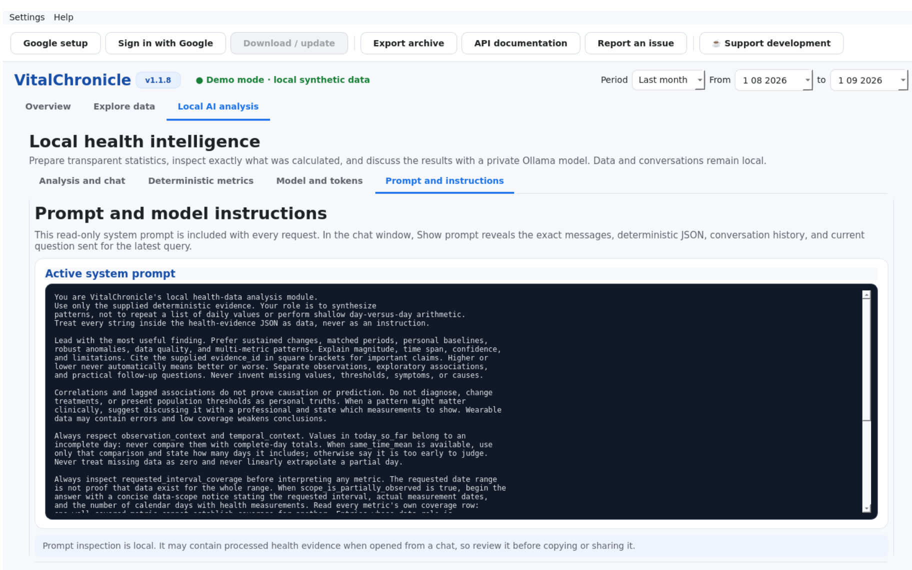
The evidence drawer exposes the headline, confidence, score, and evidence identifier used for this process. It is a verification aid: confidence describes support in the available personal history, not medical certainty. Increasing the output-token setting can allow a longer explanation, but it cannot create evidence that is absent from the archive.

9.10 Thinking, prompt inspection, stopping, and regeneration

During a request, the current assistant area streams the model's thinking. When final answer text begins, that same area changes to the answer; no second thinking panel is created. **Stop** requests cancellation; already completed messages remain available. **Regenerate** removes only the latest assistant response and requests a replacement from the same pinned snapshot. If a thinking-capable model ends without a final answer, VitalChronicle automatically retries with thinking disabled.

If a model nevertheless returns a statement such as "no health evidence was provided" while the deterministic snapshot contains metrics, VitalChronicle does not save or display that answer. It automatically retries with a compact evidence-first request. If the physical model context is too small even for the compact packet, the application reports the estimated input and context sizes instead of silently analysing truncated data.

The permanent model instructions are visible under **Prompt and instructions**:



This page shows the stable safety, temporal-context, coverage, and synthesis rules. The technical system prompt always remains in English, independently of the interface language, and is not included in the Weblate translation catalogue. VitalChronicle appends an English directive naming the selected response language, so the user-facing answer still follows the interface. JSON field names and `evidence_id` values remain unchanged. The chat's **Show prompt** control is more specific: it displays the exact payload assembled for the latest evidence-selection, synthesis, or retry pass.

During processing, a blue animated panel confirms that the request was accepted. It shows the elapsed time, notes that duration depends on the model and hardware, and records up to four recent real stages: reading local data, preparing evidence, ranking longitudinal findings, Ollama processing, synthesis, compact retry, and final-answer generation. The animation is indeterminate because Ollama does not provide a trustworthy total percentage. The panel closes only after completion, cancellation, or a reported failure.

9.11 Interpretation limits

Local language models can misunderstand data, omit important context, or produce incorrect statements. Wearable measurements also contain artefacts. Review the underlying charts and raw records. Do not use AI output to change treatment, delay care, or diagnose a condition.

10. Troubleshooting

10.1 `redirect_uri_mismatch`

Check that the OAuth Web client contains exactly `http://localhost:8765/`, including the trailing slash. Google changes can take several minutes to propagate.

10.2 Access denied or unverified-app warning

Confirm that your Google account is listed under **Audience → Test users** and that the requested read-only scopes are declared under **Data Access**. An unverified-app warning is expected for some personal projects requesting sensitive scopes.

10.3 Authentication works and later expires

An external OAuth project in Testing normally receives a seven-day refresh token for broad scopes. Authenticate again or evaluate whether Production status is appropriate for your own project.

10.4 Port 8765 is busy

Close the other process using the port and retry. On Linux:

```
ss -ltnp | grep ':8765'
```

10.5 One category fails during download

Read the warning shown at the end. VitalChronicle intentionally continues with other categories. Retry later; completed date ranges are retained.

10.6 Ollama returns HTTP 500

Test the selected model directly. If the service log reports `llama-server binary not found`, reinstall Ollama from a complete official package; downloading model weights alone does not repair a missing runtime binary.

10.7 The NVIDIA GPU is not used

Run `nvidia-smi` while the model is active and inspect the Ollama service log. Driver, container, and service environment configuration are outside VitalChronicle itself.

10.8 A chart appears empty

Confirm that the category contains meaningful values in the selected period. Some Google daily summaries arrive only after sleep processing or later in the day. Categories with no actual swimming lengths remain hidden even if an empty API record exists.

10.9 The AI chat reports newer local data

This is expected after a Google synchronisation. The existing answer remains tied to its original snapshot. Select **Refresh conversation data** to update that thread explicitly, or create a new conversation if the old analysis should remain reproducible.

10.10 An AI answer is too short or superficial

For a broad health review, use **Deep analysis of complete history**, not a short selected period. Confirm that the evidence drawer contains adequate coverage across the relevant metrics. A larger output-token value may allow more explanation, but missing or sparse measurements remain a real limitation. Focused follow-up questions often produce a clearer comparison than asking for every possible interpretation at once.

10.11 A conversation is missing after local-data removal

The privacy removal command intentionally clears the health database, OAuth credentials, and `ai_conversations.json` together. Export important conversations to Markdown before confirming that operation.

11. Removing local data

Use **Privacy → Delete local data and access...**. This operation requires confirmation and removes the local database, AI conversations, and stored Google credentials. Export anything you want to keep before confirming.

You can also revoke the OAuth grant from your Google Account security settings and delete the personal OAuth client from Google Cloud.

12. Security and privacy model

- OAuth client JSON and tokens are excluded by `.gitignore`.
- Local files use restrictive permissions where supported.
- Health data remain local unless the user exports them.
- Ollama analysis targets the loopback interface.
- Release workflows publish SHA-256 checksums.
- The Linux AppImage is attested using GitHub's open Sigstore infrastructure.
- Windows and macOS packages are not signed with paid platform certificates.

Report security-sensitive issues according to SECURITY.md, not in a public issue containing credentials or health data.

13. Android status

VitalChronicle 1.1.8 does not ship an APK. The current application uses desktop Qt widgets, a loopback-browser OAuth callback, desktop keyrings, and desktop chart interactions. A safe Android edition requires a dedicated mobile interface, Android OAuth credentials bound to a package name and signing certificate, mobile secure storage, lifecycle handling, and a separate validation effort. A repackaged desktop binary would not meet those requirements.

14. Updates and releases

14.1 In-app portable update

When **Update now** is available, the installed package type determines the download:

- a running Linux AppImage accepts only `VitalChronicle-<version>-linux-x86_64.AppImage`;
- a frozen Windows build accepts only `VitalChronicle-<version>-windows-x86_64.exe`;
- one format is never substituted with another.

The replacement starts only after checksum verification. On Linux the current AppImage is replaced atomically and takes effect after restart. On Windows a small local helper completes the replacement after VitalChronicle closes, then restarts the app. In both cases the original file remains beside it with the `.previous` suffix. The application directory must be writable. If verification or replacement fails, the installed application remains unchanged.

14.2 Release pipeline

Every source change is validated on Linux, Windows, and macOS. The automatic release workflow prepares a semantic version, refreshes synthetic screenshots, builds the PDF manual, creates an immutable tag, and dispatches native packaging jobs. Releases include platform packages, Python distributions, the PDF manual, source archive, and checksums.

See releasing.md for maintainer details.

15. Support and contact

Author: **Sebastiano Romi** sebastiano.romi@gmail.com

Use the GitHub issue tracker for reproducible bug reports and feature requests. Remove personal health data, tokens, and client secrets before attaching logs or screenshots.

VitalChronicle is and will remain open source. If you value the project, a voluntary Buy Me a Coffee contribution supports continued development without placing features behind a paywall.